บทที่ 9

พิกัดตำแหน่ง แผนที่ และเซนเซอร์อุปกรณ์

การพัฒนาแอปพลิเคชันให้สามารถโต้ตอบกับโลกกายภาพภายนอก จำเป็นต้องอาศัยกลไกการเชื่อมโยงระบบพิกัดดาวเทียม (GPS) การจัดแสดงกราฟิกแผนที่เชิงภูมิศาสตร์ และการดักรับสัญญาณเซนเซอร์ของมือถือ ในบทเรียนนี้เราจะศึกษาการจัดการระบบขออนุญาตสิทธิ์ตำแหน่งทางภูมิศาสตร์ (Location Permissions) การเข้าถึงพิกัดด้วย `expo-location`, การเรนเดอร์แผนที่ด้วย `react-native-maps` และการใช้งานเซนเซอร์วัดแรงเร่ง `expo-sensors` (Accelerometer) ในระดับมาตรฐานสากล

9.1 ระบบสิทธิ์การเข้าถึงอุปกรณ์ (Runtime Permissions)

ในการเข้าถึงทรัพยากรที่มีความปลอดภัยและข้อมูลส่วนบุคคลของผู้ใช้งาน (เช่น พิกัดแผนที่ตำแหน่ง GPS) ระบบปฏิบัติการทั้ง iOS และ Android บังคับใช้ระบบ **สิทธิ์การเข้าถึงเวลาทำงาน (Runtime Permissions)** ซึ่งหมายความว่าแอปพลิเคชันไม่สามารถดึงข้อมูลได้ทันทีเมื่อเปิดใช้งาน แต่ต้องแสดงหน้าต่างขออนุมัติสิทธิ์ (Permission Request Pop-up) จากผู้ใช้เสียก่อน

ประเภทของสิทธิ์เข้าถึงพิกัดตำแหน่ง (Location Permission Types):

1. **Foreground Location (ตำแหน่งเบื้องหน้า):** อนุญาตให้เข้าถึงข้อมูล GPS เฉพาะในขณะที่ผู้ใช้กำลังเปิดเปิดหน้าจอแอปพลิเคชันทำงานอยู่เท่านั้น (เป็นสิทธิ์หลักที่เรามักร้องขอ)
2. **Background Location (ตำแหน่งเบื้องหลัง):** อนุญาตให้แอปพลิเคชันคอยติดตามข้อมูล GPS ตลอดเวลาแม้แอปพลิเคชันจะถูกพับปิดหน้าต่างไปแล้ว (ต้องขอสิทธิ์ Foreground สำเร็จก่อนและผ่านเกณฑ์พิจารณาพิเศษจากสโตร์)

สำหรับระบบ Expo SDK 54 การขอสิทธิ์และตรวจสอบสถานะจะถูกควบคุมอย่างเป็นระบบผ่านคำสั่งสัญญา Asynchronous (Promise-based) ซึ่งส่งคืนค่าสถานะเป็นออบเจกต์ที่มีพร็อพส์ `status` บ่งชี้ผลลัพธ์:

- `'granted'`: ได้รับอนุมัติสิทธิ์สำเร็จ ดึงข้อมูลฮาร์ดแวร์ได้
- `'denied'`: ถูกปฏิเสธสิทธิ์ ไม่สามารถเข้าใช้ฮาร์ดแวร์ได้
- `'undetermined'`: ยังไม่เคยร้องขอสิทธิ์ (ผู้ใช้เปิดแอปครั้งแรก)

9.2 การเข้าถึงพิกัดตำแหน่งอุปกรณ์ด้วย expo-location

เพื่อดึงพิกัดตำแหน่งละติจูดและลองจิจูดจากอุปกรณ์เนทีฟ เราจะใช้งานไลบรารีอย่างเป็นทางการของ Expo คือ `expo-location` (Expo, 2024)

การติดตั้งไลบรารี:

```
npx expo install expo-location
```

การตั้งค่าขออนุญาตใน app.json (สำหรับ Android & iOS):

```
{
 "expo": {
  "plugins": [
   [
    "expo-location",
        {
     "locationAlwaysPermission": "อนุญาตให้แอปเข้าถึงตำแหน่งของคุณได้ตลอดเวลาเพื่อชิงก์แผนที่",
     "locationWhenInUsePermission": "อนุญาตให้เข้าถึงตำแหน่งในขณะเปิดใช้งานแอปเพื่อปักหมุดพิกัด"
        }
   ]
  ]
 }
}
```

การเขียนโค้ดดึงตำแหน่ง GPS ล่าสุด:

```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View, ActivityIndicator } from 'react-native';
import * as Location from 'expo-location';

export default function CurrentLocationApp() {
  const [location, setLocation] = useState<Location.LocationObject |
null>(null);
  const [errorMsg, setErrorMsg] = useState<string | null>(null);

  useEffect(() => {
    const requestLocation = async () => {
   // 1. ส่งคำขออนุมัติสิทธิ์เข้าถึงพิกัดตำแหน่งเบื้องหน้า (Foreground)
      let { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') {
        setErrorMsg('การเข้าถึงตำแหน่งของคุณถูกปฏิเสธ');
        return;
      }

   // 2. เมื่อได้รับอนุญาต ให้ทำการดึงพิกัดตำแหน่งละติจูดลองจิจูดปัจจุบัน
      let currentLocation = await Location.getCurrentPositionAsync({});
      setLocation(currentLocation);
```

```
    };

    requestLocation();
  }, []);

  return (
    <View style={styles.container}>
      {errorMsg ? (
        <Text style={styles.errorText}>{errorMsg}</Text>
   ) : location ? (
        <View>
          <Text style={styles.text}>Latitude:
{location.coords.latitude}</Text>
          <Text style={styles.text}>Longitude:
{location.coords.longitude}</Text>
          <Text style={styles.subText}>ระดับความแม่นยำ:
{location.coords.accuracy} เมตร</Text>
        </View>
   ) : (
        <ActivityIndicator size="large" color="#38bdf8" />
   )}
    </View>
 );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center',
backgroundColor: '#0f172a' },
  text: { fontSize: 18, color: '#ffffff', fontWeight: 'bold', marginBottom:
8 },
  subText: { fontSize: 13, color: '#94a3b8' },
  errorText: { color: '#ef4444', fontSize: 15 },
});
```

9.3 การแปลงพิกัดกลับเป็นที่อยู่ (Reverse Geocoding)

ข้อมูลละติจูดและลองจิจูด (เช่น 13.7367, 100.5231) เป็นข้อมูลที่ยากต่อการทำความเข้าใจของผู้ใช้ทั่วไป ไลบรารี **expo-location** จึงได้นำเสนอคุณสมบัติ **Reverse Geocoding** ซึ่งทำหน้าที่แปลงพิกัดทางภูมิศาสตร์ตัวเลขให้กลายเป็นออบเจกต์ที่อยู่จริงของที่ตั้งปลายทาง เช่น ชื่ออาคาร, เลขที่ถนน, อำเภอ, จังหวัด และรหัสไปรษณีย์

ตัวอย่างคำสั่งแปลงพิกัด:

```
const getAddressFromCoords = async (latitude: number, longitude: number)
=> {
```

```
  try {
  // ส่งละติจูดและลองจิจูดไปถอดข้อมูลภูมิภาค
    const addressResponse = await Location.reverseGeocodeAsync({
      latitude: latitude,
      longitude: longitude,
   });

    if (addressResponse.length > 0) {
      const address = addressResponse[0];
   // ตัวอย่างการดึงฟิลด์ข้อมูลที่อยู่
      console.log(`ที่อยู่: ถนน ${address.street}, อำเภอ ${address.district}, จังหวัด
${address.region}`);
    }
  } catch (error) {
    console.error('การแปลงที่อยู่ล้มเหลว:', error);
  }
};
```

9.4 การติดตั้งและจัดแสดงแผนที่ด้วย react-native-maps

ในการเรนเดอร์ภาพแผนที่เชิงโต้ตอบ (Interactive Maps) ที่ผู้ใช้งานสามารถย่อขยาย ดึงแผนที่ และมองเห็นความตื้นลึกของระดับสิ่งปลูกสร้างอย่างไหลลื่น เราจะติดตั้งใช้งานไลบรารีภายนอกยอดนิยมคือ **`react-native-maps`** (React Native Maps, 2024)

การติดตั้ง:

```
npx expo install react-native-maps
```

คอมโพเนนต์หลักที่ใช้งาน:

• **`<MapView>`**: คอนเทนเนอร์หลักที่ใช้สำหรับจำลองเรนเดอร์แผนที่ภูมิศาสตร์ลงบนหน้าจออุปกรณ์

• **`<Marker>`**: คอมโพเนนต์ลูกย่อยที่ถูกเรนเดอร์ไว้ข้างใน MapView เพื่อแสดงสัญลักษณ์หมุดปัก (Map Pin) ณ จุดพิกัดใดพิกัดหนึ่งโดยเฉพาะ

9.5 การควบคุมแผนที่ (Region) และการใช้งาน Marker

การกำหนดมุมกล้องและพิกัดซูมบนคอมโพเนนต์ **`<MapView>`** จะถูกควบคุมผ่านพร็อพส์ **`region`** ซึ่งจะรับออบเจกต์ประกอบด้วยข้อมูล 4 ส่วนหลัก:

1. **`latitude`**: ละติจูดของพิกัดที่จะให้เป็นกึ่งกลางหน้าจอแผนที่
2. **`longitude`**: ลองจิจูดของพิกัดที่จะให้เป็นกึ่งกลางหน้าจอแผนที่

3. `latitudeDelta`: ระยะการซูมในแนวตั้ง (ค่ายิ่งน้อย ยิ่งซูมเข้าลึก มองเห็นรายละเอียดถนนและซอยชัดเจน)

4. `longitudeDelta`: ระยะการซูมในแนวนอน (ค่ายิ่งน้อย ยิ่งซูมเข้าลึก)

```
import React from 'react';
import { StyleSheet, View } from 'react-native';
import MapView, { Marker } from 'react-native-maps';

export default function SimpleMap() {
  return (
    <View style={styles.container}>
      <MapView
        style={styles.map}
    // กำหนดตำแหน่งศูนย์กลางกล้องแผนที่เริ่มต้น (กรุงเทพมหานคร)
        initialRegion={{
          latitude: 13.7563,
          longitude: 100.5018,
          latitudeDelta: 0.05,
          longitudeDelta: 0.05,
        }}
      >
        {/* ปักหมุดตำแหน่ง ณ เสาชิงช้า */}
        <Marker
          coordinate={{ latitude: 13.7519, longitude: 100.5012 }}
          title="เสาชิงช้า"
          description="จุดเช็คอินใจกลางกรุงเทพมหานคร"
    />
      </MapView>
    </View>
 );
}

const styles = StyleSheet.create({
  container: { flex: 1 },
  map: { width: '100%', height: '100%' },
});
```

9.6 การใช้งานเซนเซอร์ตรวจความเร่งด้วย expo-sensors (Accelerometer)

อุปกรณ์สมาร์ทโฟนสมัยใหม่ทุกเครื่องจะมีชิปเซนเซอร์ภายในเครื่องสำหรับวัดมุม การหมุน และความเร่ง ไลบรารี `expo-sensors` (Expo, 2024) ช่วยให้ผู้พัฒนาสามารถอ่านค่าพฤติกรรมกายภาพของมือถือได้ง่าย

การดาวน์โหลดแพ็คเกจ:

```
npx expo install expo-sensors
```

การใช้งาน Accelerometer (เซนเซอร์วัดแรงเร่ง):

เซนเซอร์นี้จะคอยส่งชุดตัวเลขตามแกนพิกัดสามมิติ (x, y, z) อิงตามแรงโน้มถ่วงและการเคลื่อนที่จริงของตัวเครื่อง:

- **แกน x:** เอียงเครื่องไปซ้าย/ขวา
- **แกน y:** เอียงเครื่องไปหน้า/หลัง
- **แกน z:** แรงดึงแนวตั้งของการถือเครื่อง

```
import React, { useState, useEffect } from 'react';
import { StyleSheet, Text, View } from 'react-native';
import { Accelerometer } from 'expo-sensors';

export default function MotionSensorApp() {
  const [data, setData] = useState({ x: 0, y: 0, z: 0 });
  const [subscription, setSubscription] = useState<any>(null);

 // 1. สั่งเปิดรับฟังการเปลี่ยนแปลงข้อมูลจากชิปเซนเซอร์
  const subscribe = () => {
    setSubscription(
      Accelerometer.addListener((accelerometerData) => {
        setData(accelerometerData);
      })
  );
  // ตั้งรอบเวลาตรวจจับข้อมูลทุก ๆ 200 มิลลิวินาที
    Accelerometer.setUpdateInterval(200);
  };

 // 2. ยกเลิกการดักฟลักซ์ข้อมูลเพื่อเซฟแบตเตอรี่เครื่อง
  const unsubscribe = () => {
    subscription && subscription.remove();
    setSubscription(null);
  };

  useEffect(() => {
    subscribe();
  // คืนค่าฟังก์ชัน Unsubscribe สำหรับ Cleanup ตอนปิดหน้าจอป้องกัน RAM รั่ว
    return () => unsubscribe();
  }, []);

  return (
    <View style={styles.container}>
      <Text style={styles.title}>ค่าความเร่ง Accelerometer</Text>
      <Text style={styles.coord}>X: {data.x.toFixed(3)}</Text>
      <Text style={styles.coord}>Y: {data.y.toFixed(3)}</Text>
```

```
      <Text style={styles.coord}>Z: {data.z.toFixed(3)}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center', backgroundColor: '#0f172a' },
  title: { fontSize: 18, color: '#38bdf8', fontWeight: 'bold', marginBottom: 12 },
  coord: { fontSize: 24, color: '#ffffff', fontFamily: 'monospace', marginVertical: 4 },
});
```

9.7 แนวทางการเขียนดักจับเหตุการณ์ปฏิเสธสิทธิ์และคืนค่าทรัพยากร (Permission Denied & Memory Cleanups)

กฎเหล็กสำคัญสองประการในการป้องกันไม่ให้โมบายแอปพลิเคชันเกิดอาการค้างหรือกินหน่วยความจำมหาศาลจนแอปเด้งดับ มีดังนี้:

1. การจัดการกรณีสิทธิ์การเข้าถึงถูกปฏิเสธ (Permission Rejection Fallback)

หากผู้ใช้งานไม่ยินยอมเปิด GPS ให้แอปพลิเคชัน ผู้พัฒนาไม่ควรปล่อยให้แอปพลิเคชันเรนเดอร์แผนที่ว่างเปล่าหรือค้างหน้าจอขาว แต่ต้องสลับ UI มาแสดงกล่องข้อความอธิบายความจำเป็นของการใช้พิกัด พร้อมดีไซน์ปุ่มกด "ตั้งค่าอุปกรณ์ (Open Settings)" เพื่อนำทางด่วนไปหน้าตั้งค่าระบบปฏิบัติการของเครื่อง:

```
import * as Linking from 'expo-linking';
import { Button, View, Text } from 'react-native';

const handleOpenSettings = () => {
 // สั่งสั่งระบบทางตรงไปที่แอปพลิเคชันตั้งค่าสิทธิ์บนโทรศัพท์
  Linking.openSettings();
};
```

2. การล้างและปิดสตรีมเซนเซอร์ (Memory Cleanups)

เซนเซอร์ต่าง ๆ (เช่น Accelerometer, Gyroscope) ทำการสตรีมฟีดค่าพารามิเตอร์ขึ้นมาอัปเดตตลอดเวลา หากเปิดทริกเกอร์ฟังข้อมูลไว้แต่มีการเปลี่ยนไปหน้าอื่น สตรีมนี้จะยังทำงานค้างอยู่เบื้องหลัง ดึงแรมและกินแบตเตอรี่จนเครื่องร้อนจัด ผู้พัฒนาจำเป็นต้องคืนค่าและระงับการทำงาน (Unsubscribe) ใน Cleanup Function ของ React `useEffect` เสมอ

สรุปท้ายบทเรียน

ระบบสิทธิ์การเข้าถึงอุปกรณ์ (Runtime Permissions): การรักษาความปลอดภัยด้านความเป็นส่วนตัวบังคับให้แอปพลิเคชันต้องขอสิทธิ์เข้าถึงพิกัด GPS ระหว่างรันแอปผ่านระบบ Runtime Permissions ซึ่งสามารถขอสิทธิ์แบบเบื้องหน้า (Foreground) หรือเบื้องหลัง (Background) โดยระบบ Expo SDK 54 จัดการสถานะสิทธิ์การเข้าถึงส่งคืนค่าเป็นสัญญา Asynchronous ที่ส่งผลลัพธ์หลักเป็น granted (ได้รับสิทธิ์), denied (ถูกปฏิเสธ) หรือ undetermined (ยังไม่ร้องขอ)

การเข้าถึงพิกัดตำแหน่งอุปกรณ์ด้วย expo-location: การค้นหาพิกัดละติจูดและลองจิจูดของโทรศัพท์ใช้ไลบรารี expo-location ซึ่งต้องลงทะเบียนขออนุญาตในไฟล์ตั้งค่าโครงการ app.json ก่อน และใช้คำสั่ง requestForegroundPermissionsAsync ร่วมกับ getCurrentPositionAsync ในการดักจับพิกัดละติจูดและลองจิจูดปัจจุบัน รวมถึงระดับค่าความแม่นยำของจุดปักพิกัดสัญญาณดาวเทียมเพื่อนำไปใช้แสดงผลต่อ

การแปลงพิกัดกลับเป็นที่อยู่ (Reverse Geocoding): เนื่องจากพิกัดตัวเลขทางภูมิศาสตร์เข้าใจยากสำหรับผู้ใช้งานทั่วไป expo-location ได้จัดเตรียมความสามารถ Reverse Geocoding ผ่านฟังก์ชัน reverseGeocodeAsync ในการเปลี่ยนค่าละติจูดและลองจิจูดเป็นที่อยู่จริงของอุปกรณ์ เช่น ชื่อถนน ตำบล อำเภอ จังหวัด และรหัสไปรษณีย์ เพื่อแปลผลพิกัดเป็นข้อมูลที่แสดงผลแบบสื่อสารเข้าใจได้ง่ายบนอินเทอร์เฟซ

การติดตั้งและจัดแสดงแผนที่ด้วย react-native-maps: การแสดงแผนที่เชิงภูมิศาสตร์แบบโต้ตอบได้เต็มรูปแบบจะใช้ไลบรารี react-native-maps (React Native Maps, 2024) โดยประกอบไปด้วยคอมโพเนนต์หลัก ได้แก่ MapView ในการทำหน้าที่วาดหน้าต่างแผนที่ลงบนเฟรมหน้าต่างแอปพลิเคชัน และคอมโพเนนต์ย่อยอย่าง Marker เพื่อระบุพิกัดปักหมุดสีแดง (Map Pin) ณ ละติจูดและลองจิจูดเป้าหมายพร้อมเขียนบรรยายรายละเอียดกำกับ

การควบคุมแผนที่ (Region) และการใช้งาน Marker: การควบคุมการแสดงผลและปรับมุมกล้องแผนที่จะทำงานผ่านคุณสมบัติ Region ใน MapView ประกอบด้วยค่าศูนย์กลางละติจูด-ลองจิจูด และค่าความกว้างของการซูมอย่าง latitudeDelta และ longitudeDelta ซึ่งค่ายิ่งน้อยจะยิ่งซูมเข้าสู่ผิวถนนได้อย่างชัดเจน และเราสามารถปักหมุด Marker ระบุข้อมูล coordinate รวมถึงพิมพ์คำอธิบายหัวข้อจุดเช็คอินต่าง ๆ ได้อย่างอิสระ

การใช้งานเซนเซอร์ตรวจความเร่งด้วย expo-sensors: การเขียนรหัสรับรู้พฤติกรรมการเคลื่อนไหวจริงของโทรศัพท์ใช้ไลบรารี expo-sensors ในการอ่านค่าจากชิป Accelerometer เพื่อฟีดพารามิเตอร์ความเร่งสามแกน (x, y, z) สะท้อนแรงโน้มถ่วงเมื่อพลิก เอียง หรือขยับโทรศัพท์

โดยต้องระบุช่วงความถี่ในการตรวจสอบผ่าน setUpdateInterval เพื่อสตรีมข้อมูลความเคลื่อนไหวมาแสดงผลแบบเรียลไทม์

แนวทางการเขียนดักจับเหตุการณ์ปฏิเสธสิทธิ์และคืนค่าทรัพยากร: โครงสร้างความปลอดภัยของแอปพลิเคชันที่ดีต้องเตรียมระบบรองรับกรณีผู้ใช้ปฏิเสธสิทธิ์โดยใช้ Linking.openSettings() เพื่อพาลัดเข้าหน้าการตั้งค่าเครื่อง และที่สำคัญที่สุดคือการป้องกันปัญหาระบบเมมโมรี่รั่วไหล (Memory Leak) จากฟีดข้อมูลเซนเซอร์ โดยการสั่งยกเลิกติดตามเซนเซอร์ (Unsubscribe) ในส่วนของ Cleanup Function ของ React useEffect ก่อนเปิดหน้าจอแอปพลิเคชันถัดไป

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความเข้าใจเกี่ยวกับกลไกขออนุญาตเข้าใช้ฮาร์ดแวร์ (Location Permissions), การค้นหาพิกัดดาวเทียมปัจจุบันและการแปลงค่าเป็นที่อยู่ภาษาไทย (`expo-location`), การควบคุมมุมซูมและปักหมุดบนแผนที่ (`react-native-maps`), และการรับฟีดข้อมูลเซนเซอร์เคลื่อนไหวรวมถึงกลไกป้องกันหน่วยความจำรั่วไหล (`expo-sensors`) ในระบบปฏิบัติการโทรศัพท์มือถือ

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดอธิบายความแตกต่างระหว่างสิทธิ์การเข้าถึงตำแหน่งชนิด Foreground และ Background Location ได้ถูกต้อง?

ก. Foreground ใช้ได้เฉพาะในระบบ Android ส่วน Background ใช้ได้เฉพาะระบบ iOS

ข. Foreground อนุญาตให้อ่าน GPS เฉพาะขณะเปิดใช้งานแสดงผลหน้าจอแอปพลิเคชัน ส่วน Background อนุญาตให้ติดตามข้อมูลได้ตลอดเวลาแม้แอปจะพับปิดไปเบื้องหลัง

ค. Foreground บังคับใช้เฉพาะสัญญาณอินเทอร์เน็ต WiFi ส่วน Background ใช้กับเครือข่ายดาวเทียมเท่านั้น

ง. Foreground ใช้ดึงข้อมูลพิกัดความเร็วเครื่อง ส่วน Background ใช้บันทึกภาพถ่ายสถานที่

ข้อ 2: เมื่อต้องการขอสิทธิ์เข้าใช้งานพิกัดตำแหน่งเป็นครั้งแรกในขณะที่ผู้เรียนกำลังรันแอป ผู้พัฒนาต้องเรียกใช้งานคำสั่งใดจากไลบรารี `expo-location`?

ก. `Location.getCurrentPositionAsync()`

ค. `Location.requestForegroundPermissionsAsync()`

ค. `Location.reverseGeocodeAsync()`

ง. `Location.openSettings()`

ข้อ 3: เมธอดใดของ `expo-location` ที่ใช้ทำหน้าที่แปลงตัวเลขพิกัดละติจูดและลองจิจูด (Latitude, Longitude) ให้กลายเป็นที่อยู่สิ่งปลูกสร้างหรือถนนจริงภาษาไทย?

ก. `Location.getCurrentPositionAsync`

ข. `Location.requestBackgroundPermissionsAsync`

ค. `Location.reverseGeocodeAsync`

ง. `Location.geocodeAsync`

ข้อ 4: ข้อใดบ่งบอกถึงวัตถุประสงค์หลักของการกำหนดตัวแปร `latitudeDelta` และ `longitudeDelta` ในพร็อพส์ `region` ของคอมโพเนนต์ `<MapView>`?

ก. กำหนดความเร็วในการเลื่อนหมุนแผนที่เมื่อผู้ใช้ใช้นิ้วปัด

ข. กำหนดระดับช่วงการมองเห็นหรือระยะการซูมเข้า/ซูมออกของกล้องแผนที่ (ค่ายิ่งน้อย ยิ่งซูมขยายเข้าลึก)

ค. กำหนดพิกัดทางภูมิศาสตร์ของทิศเหนือและทิศใต้เพื่อล้างค่าขอบเขต

ง. กำหนดมุมเอียงแบบสามมิติของตึกระฟ้าบนแผนที่

ข้อ 5: หากต้องการปักหมุดจุดสัญลักษณ์ เช่น รูปภาพเข็มหมุดสีแดง เพื่อเน้นจุดพิกัดร้านค้าหรือบ้านในแผนที่ `<MapView>` ผู้พัฒนาต้องเรียกใช้งานคอมโพเนนต์ลูกตัวใด?

ก. `<MapView.Circle>`

ข. `<Marker>`

ค. `<View>`

ง. `<Overlay>`

ข้อ 6: พร็อพส์ (Props) ตัวใดที่เป็นตัวคุมละติจูดและลองจิจูดของตำแหน่งหมุดในคอมโพเนนต์ `<Marker>`?

ก. `region`

ข. `initialRegion`

ค. `coordinate`

ง. `center`

ข้อ 7: ชิปเซนเซอร์ "Accelerometer" ในระบบปฏิบัติการโทรศัพท์เคลื่อนที่ คอยส่งมอบชุดตัวเลขชนิดใดเพื่อรายงานพฤติกรรมทางกายภาพของตัวเครื่อง?

ก. ระบุค่าความเร็วการเชื่อมต่อสัญญาณอินเทอร์เน็ตเฉลี่ย 3 ทิศทาง

ข. ระบุค่าความเร่งตามแกนสามมิติ (x, y, z) สัมพันธ์กับแรงโน้มถ่วงและการสั่นสะเทือนของเครื่อง

ค. ระบุทิศทางมุมองศาของเข็มทิศขั้วแม่เหล็กโลกทางภูมิศาสตร์

ง. ระบุปริมาณความกดอากาศและความชื้นสัมพัทธ์ในห้อง

ข้อ 8: เมธอดใดของเซนเซอร์ `Accelerometer` ที่ใช้สำหรับลงทะเบียนและเริ่มต้นดักจับการสั่นสะเทือนเครื่องแบบเรียลไทม์?

ก. `Accelerometer.setUpdateInterval()`

ข. `Accelerometer.addListener()`

ค. `Accelerometer.remove()`

ง. `Accelerometer.removeAllListeners()`

ข้อ 9: เหตุใดการเขียนฟังก์ชัน cleanup คืนค่า `subscription.remove()` ภายใน `useEffect` จึงมีความสำคัญระดับสูงสุดเมื่อเรียกใช้งานเซนเซอร์ในแอป React Native?

ก. เพื่อไม่ให้ตัวแปรของสไตล์ชีทพังเมื่อเข้าสู่โหมดมืด

ข. เพื่อปิดกระบวนการรับฟีดข้อมูลที่ค้างคาอยู่ในแรม (RAM) ช่วยเซฟแบตเตอรี่เครื่องและป้องกันปัญหาเครื่องค้างเนื่องจากหน่วยความจำรั่วไหล (Memory Leak)

ค. เพื่อสั่งดาวน์โหลดฟอนต์ Prompt ใหม่เข้ามาทำงานทดแทน

ง. เพื่อสุ่มเปลี่ยนรูปภาพแผนที่ให้กลายเป็นสกินดาวเทียม

ข้อ 10: หากผู้ใช้เผลอกดปฏิเสธไม่ยอมอนุมัติให้แอปเข้าถึง Location และเราต้องการเขียนปุ่มลัดเพื่ออำนวยความสะดวกให้นำทางผู้ใช้ไปยังหน้าตั้งค่าสิทธิ์บนโทรศัพท์โดยตรง เราควรเลือกใช้เครื่องมือใดของ Expo?

ก. `Linking.openURL()`

ข. `Linking.openSettings()`

ค. `Location.requestForegroundPermissionsAsync()`

ง. `Location.getCurrentPositionAsync()`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ทั้ง iOS และ Android ต่างสนับสนุนโมเดล Foreground และ Background ทั้งคู่

• **ข. ถูก:** Foreground จะดึงพิกัดได้เฉพาะขณะผู้ใช้เปิดแอปค้างไว้บนจอ ส่วน Background ดึงได้ตลอดเวลาแม้แอปพลิเคชันจะรันอยู่เงียบ ๆ ด้านหลัง

• **ค. และ ง. ผิด:** ไม่เกี่ยวข้องกับคลื่นสัญญาณ และไม่ใช่เครื่องมือเปิดกล้องบันทึกภาพถ่าย

เฉลย ข้อ 2: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `getCurrentPosition` ใช้ดึงค่าพิกัด แต่จะรันพังทันทีหากยังไม่ได้รับสิทธิ์อนุมัติ

• **ข. ถูก:** คำสั่งที่มี `request` นำหน้า เช่น `Location.requestForegroundPermissionsAsync()` จะส่งสัญญาณเรียกหน้าต่าง Pop-up ดั้งเดิมของระบบปฏิบัติการขึ้นมาสัมภาษณ์สิทธิ์จากผู้ใช้งานแอป

• **ค. และ ง. ผิด:** ใช้สำหรับแปลงค่าที่อยู่ และลิงก์ไปหน้าตั้งค่า ไม่ใช่หน้าต่างกล่องป๊อปอัพตัวเริ่มต้นขอสิทธิ์

เฉลย ข้อ 3: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ดึงพิกัดตัวเลขไม่ใช่ที่อยู่

• **ข. ผิด:** เป็นคำสั่งขอสิทธิ์ตำแหน่งเบื้องหลัง

• **ค. ถูก:** Reverse Geocoding คือเมธอดแปลงจากพิกัด (Coords) กลับคืนเป็น Address (ที่อยู่อักษร)

• **ง. ผิด:** GeocodeAsync จะทำงานกลับทิศกัน (แปลงจากที่อยู่ตัวหนังสือให้กลายเป็นเลขพิกัดละติจูดลองจิจูด)

เฉลย ข้อ 4: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่เกี่ยวกับสปีดการปัดนิ้ว และไม่ใช่ระบบจัดองศาสามมิติตึก

• **ข. ถูก:** Delta เป็นตัวชี้วัดความกว้างของช่วงพิกัดที่กล้องจะเก็บภาพมาเรนเดอร์ ตัวเลขยิ่งน้อยแสดงว่าสัดส่วนกล้องจะขยายโฟกัสซูมเข้าไปใกล้ขอบทางเดินได้ลึกและละเอียดขึ้น

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** Circle วาดพื้นที่วงกลมครอบรัศมีบนแผนที่

• **ข. ถูก:** คอมโพเนนต์ `<Marker>` เป็นหัวใจของการระบุจุดสำคัญลงบนแผนที่

• **ค. และ ง. ผิด:** View เป็นบล็อกจัดหน้าตาปกติ และ Overlay ใช้ระบุแผ่นพื้นที่ทับซ้อน

เฉลย ข้อ 6: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. และ ข. ผิด:** เป็นตัวคุมแผนที่ MapView ทั้งหมด ไม่ใช่ขอบของหมุดย่อยตัวเดียว

• **ค. ถูก:** พร็อพส์ `coordinate` ของ Marker รับอ็อบเจกต์ที่มีโครงสร้าง `{ latitude: number, longitude: number }` เพื่อนำเข็มหมุดปักตรงจุด

• **ง. ผิด:** ไม่มีคีย์ชื่อ center ในข้อกำหนดมาตรฐาน

เฉลย ข้อ 7: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่สามารถอ่านค่าเน็ต หรือความกดชื้นอากาศในเครื่องได้ และทิศทางแม่เหล็กโลกเป็นหน้าที่ของชิป Magnetometer (เข็มทิศ)

• **ข. ถูก:** เซนเซอร์วัดแรงเร่ง Accelerometer จับแรงสั่นสะเทือนรอบแกน X (ซ้ายขวา) แกน Y (หน้าหลัง) และแกน Z (แรงโน้มถ่วงแนวตั้ง) เพื่อนำมารายงานพฤติกรรมการขยับเครื่อง

เฉลย ข้อ 8: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `setUpdateInterval` คุมระยะห่างความเร็วการยิงค่า ไม่ใช่การเริ่มสตรีมรับฟัง

• **ข. ถูก:** `addListener` จะเป็นการสมัคร (Subscribe) รับฟังฟลักซ์ข้อมูลจากเซนเซอร์ โดยจะส่ง callback ผลลัพธ์กลับมาเมื่อตรวจพบการเปลี่ยนแปลงตำแหน่ง

• **ค. และ ง. ผิด:** ใช้สำหรับระงับการทำงาน

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** ไม่ได้เกี่ยวกับโหมดสี ฟอนต์อักษร หรือสกินแผนที่ผ่านดาวเทียม

• **ข. ถูก:** การสตรีมฟีดเซนเซอร์เป็นงานที่ค้างในระบบเบื้องหลังของฮาร์ดแวร์ หากไม่ยอม Unsubscribe ตอนปิดหน้าจอนั้น ๆ หรือปิดแอป สตรีมจะดูดแรมและประมวลผลตลอดไป ส่งผลให้แบตเตอรี่เครื่องเสื่อมไว ร้อนจัด และแอปเด้งดับ

เฉลย ข้อ 10: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `openURL` ใช้ลิงก์ไปหน้าเว็บไซต์ภายนอก

• **ข. ถูก:** `Linking.openSettings()` เป็นคำสั่งลัดพาตัวข้ามระบบ Sandbox ของแอปเพื่อไปเปิดหน้าควบคุมสิทธิ์เข้าถึงของแอปนั้น ๆ ในเมนูตั้งค่าหลักของสมาร์ทโฟน สะดวกต่อการให้ผู้ใช้เปิดสิทธิ์ทดแทน

• **ค. และ ง. ผิด:** เป็นเพียงคำสั่งขอสิทธิ์และอ่านค่าพิกัดปกติ ซึ่งจะรันไม่สำเร็จหากผู้ใช้ปิดสิทธิ์ไปก่อนหน้าแล้ว

ปฏิบัติการท้ายบทเรียน: การพัฒนาแอปพลิเคชันปักหมุดแผนที่และสั่นสะเทือนจดจำตำแหน่ง

ปฏิบัติการนี้จะฝึกทักษะการรวบรวมฟังก์ชันระดับฮาร์ดแวร์ของโทรศัพท์มือถือเข้าหากัน โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **"ระบบแผนที่นำทางและสั่นสะเทือนจดจำพิกัด (Interactive Location Map Tracker)"** ซึ่งจะร้องขอสิทธิ์ดาวเทียม ดึงตำแหน่ง GPS ของเครื่องมาประมวลผล ถอดรหัสแปลงกลับเป็นชื่อสถานที่จริง แสดงผลแผนที่เชิงโต้ตอบแบบซูมลึก ปักหมุดตำแหน่งปัจจุบัน และผูกเซนเซอร์ความเร่งตรวจจับความเคลื่อนไหว (Accelerometer) เมื่อผู้ใช้ทำมุมเอียงหรือเขย่าเครื่อง เพื่อบังคับมุมกล้องแผนที่ให้ดีดสไลด์กลับมาโฟกัสพิกัดผู้ใช้ปัจจุบันแบบไดนามิก

1. วัตถุประสงค์การเรียนรู้

1. สามารถเขียนรหัสควบคุมระบบสิทธิ์การเข้าถึงพิกัดตำแหน่ง (Runtime Location Permission) และเขียนระบบป้องกันเมื่อผู้ใช้ปฏิเสธสิทธิ์
2. สามารถใช้คุณสมบัติ Geocoding แปลงละติจูดลองจิจูดให้ออกมาเป็นข้อมูลตำบลและอำเภอจริงเพื่อจัดแสดงผลบนหน้าจอ
3. สามารถกำหนดคอนเทนเนอร์แผนที่ `<MapView>` ร่วมกับสัญลักษณ์ `<Marker>` โดยตั้งค่าระยะซูม Delta ที่เหมาะสมคมชัด
4. สามารถเชื่อมโยงการทำงานร่วมกับเซนเซอร์ความเร่ง `Accelerometer` เพื่อสั่งขยับพิกัดกล้องแผนที่ตามการสั่นเครื่องได้สำเร็จ
5. สามารถเขียนโค้ดระงับสตรีมเซนเซอร์ (Cleanup listeners) ได้ถูกต้องเพื่อประสิทธิภาพของทรัพยากรระบบ

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องพัฒนาแอปพลิเคชันระบบแผนที่และพิกัดตำแหน่งส่วนบุคคลตามเงื่อนไขต่อไปนี้:

1. **สิทธิ์และการเข้าถึงพิกัดตำแหน่ง (Permissions & GPS Tracking):**

• ทันทีที่แอปเปิดทำงาน ต้องทำการขอสิทธิ์ Foreground Location

• หากสิทธิ์ถูกปฏิเสธ (Denied) ให้แสดงหน้าต่างเตือนภัยปุ่มสีแดงระบุข้อความ "กรุณาเปิดรับสิทธิ์ตำแหน่งเพื่อแสดงแผนที่" พร้อมปุ่มกดดึงคำสั่ง `Linking.openSettings()` เพื่ออำนวยความสะดวกให้ผู้ใช้เข้าหน้าระบบควบคุมสิทธิ์เครื่องได้ทันที

• หากสิทธิ์ได้รับอนุมัติ ให้ทำพิกัดละติจูดลองจิจูดปัจจุบัน แปลงที่อยู่อักษรย้อนกลับด้วย `reverseGeocodeAsync` และนำข้อมูลชื่อถนน/จังหวัด แสดงเป็นแผงป้ายชื่อด้านบนสุด

2. **การจัดแสดงภาพแผนที่แบบลึกซูม (Map Rendering):**

• แสดงแผงแผนที่แบบเต็มจอในบริเวณด้านล่าง โดยมีศูนย์กลางอยู่ที่ตำแหน่งละติจูดและลองจิจูดของผู้ใช้ปัจจุบัน

• ตั้งค่าระยะซูมเข้าลึก `latitudeDelta: 0.01` และ `longitudeDelta: 0.01` เพื่อให้เห็นรายละเอียดหมู่บ้านและตรอกซอย

• เรนเดอร์คอมโพเนนต์ `<Marker>` ปักหมุดเข็มสีสว่างไว้ที่จุดพิกัดปัจจุบัน พร้อมเขียนป้ายทูลทิป (Tooltip) ระบุข้อความ "ตำแหน่งของคุณในขณะนี้"

3. **ระบบสั่นสะเทือนจัดศูนย์กลาง (Motion Sensor Integration):**

• สมัครสตรีมสัญญาณข้อมูลเซนเซอร์จากเครื่องมือ `Accelerometer` โดยอัปเดตแกนความเร่ง x, y, z

• กำหนดให้หากผู้ใช้งานเขย่าตัวเครื่องจำลอง หรือเอียงเครื่องทำมุมความเร่ง (เช่น ค่าสัมบูรณ์แกน `x` หรือ `y` มีค่าความแรงเกินกว่า 1.8) ให้แผนที่สไลด์ (Animate) นำศูนย์กลางกล้องแผนที่วิ่งกลับมาโฟกัสพิกัดเริ่มต้นปัจจุบันของผู้ใช้โดยอัตโนมัติ (เสมือนการสั่ง Center Map ด้วยการเขย่า)

4. **ความมั่นคงในการล้างทรัพยากร (Memory Safety Cleanup):**

• ผู้เรียนต้องเขียนส่งมอบฟังก์ชัน Unsubscribe เพื่อยกเลิกลบผู้ฟังค่า Accelerometer ออกจาก RAM ของเครื่องเมื่อคอมโพเนนต์ถูกปิดตัวลง

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งไลบรารีตำแหน่ง แผนที่ และเซนเซอร์

รันคำสั่งติดตั้งคลังฮาร์ดแวร์ของ Expo และ React Native เพื่อใช้งานในโครงการ:

```
npx expo install expo-location react-native-maps expo-sensors
```

ขั้นตอนที่ 2: ตั้งค่าสิทธิ์และขอบเขตใน `app.json`

เปิดไฟล์ app.json และเขียนเพิ่มค่าสิทธิ์ความปลอดภัยลงในคีย์ plugins:

```
{
 "expo": {
  "plugins": [
   [
    "expo-location",
        {
     "locationWhenInUsePermission": "เราจำเป็นต้องเข้าถึงพิกัด GPS เพื่อแสดงจุดสีน้ำเงินบนแผนที่ของคุณ"
        }
   ]
  ]
 }
}
```

ขั้นตอนที่ 3: ออกแบบระบบตรวจสอบสิทธิ์และดักจับการตั้งค่าเครื่อง

เขียนโค้ดตรวจสอบเงื่อนไขและส่งลิงก์ Settings ด้วยคำสั่ง Linking:

```
import * as Linking from 'expo-linking';

// ปุ่มกดนำทางข้ามระบบ Sandbox
<TouchableOpacity
  style={styles.settingsButton}
  onPress={() => Linking.openSettings()}
>
  <Text style={styles.buttonText}>เปิดตั้งค่าสิทธิ์สิทธิ์บนระบบเครื่อง</Text>
</TouchableOpacity>
```

ขั้นตอนที่ 4: เชื่อมโยง Accelerometer เข้าหา MapView อ้างอิงด้วย Ref

เพื่อสั่งสั่งกล้องของแผนที่เลื่อนไปมาอย่างนุ่มนวล เราจะสร้างตัวแปรอ้างอิง mapRef และสั่งการผ่านคำสั่ง animateToRegion:

```
const mapRef = React.useRef<MapView | null>(null);

// หากแรงเหวี่ยงเซนเซอร์เขย่าเกินพิกัด ให้เลื่อนศูนย์กลางกลับมา
if (Math.abs(x) > 1.8 || Math.abs(y) > 1.8) {
  mapRef.current?.animateToRegion({
    latitude: currentCoords.latitude,
    longitude: currentCoords.longitude,
    latitudeDelta: 0.01,
    longitudeDelta: 0.01,
  }, 1000); // ดีดกล้องสไลด์นุ่มนวลในเวลา 1 วินาที
}
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถคัดลอกรหัสต้นฉบับสมบูรณ์ด้านล่างนี้ บันทึกทับในไฟล์หลักของโครงการ (เช่น `app/index.tsx`) เพื่อรันสตรีมนำพิกัดดาวเทียมขึ้นระบบแผนที่พร้อมระบบสั่นตรวจจับการเขย่า:

```
import React, { useState, useEffect, useRef } from 'react';
import {
  StyleSheet,
  Text,
  View,
  ActivityIndicator,
  TouchableOpacity,
  Alert,
  SafeAreaView
} from 'react-native';
import * as Location from 'expo-location';
import MapView, { Marker } from 'react-native-maps';
import { Accelerometer } from 'expo-sensors';
import * as Linking from 'expo-linking';

// โมเดลเก็บข้อมูลพิกัด
interface Coords {
  latitude: number;
  longitude: number;
}

export default function InteractiveLocationTracker() {
  const [hasPermission, setHasPermission] = useState<boolean |
null>(null);
  const [currentCoords, setCurrentCoords] = useState<Coords | null>(null);
  const [address, setAddress] = useState<string>('กำลังค้นหาที่อยู่ของคุณ...');
  const [isLoading, setIsLoading] = useState<boolean>(true);

 // ตัวแปรเก็บค่าพิกัดความเร่งเซนเซอร์
  const [accelerometerData, setAccelerometerData] = useState({ x: 0, y: 0,
z: 0 });
  const accelerometerSubscription = useRef<any>(null);

 // ตัวอ้างอิงออบเจกต์ MapView เพื่อส่งคำสั่งจัดเฟรมกล้อง
  const mapRef = useRef<MapView | null>(null);

  useEffect(() => {
    setupLocationAndSensors();

  // คืนค่าฟังก์ชัน Cleanup เมื่อถอนหน้าจอป้องกัน RAM รั่ว
    return () => {
```

```
      stopAccelerometerListening();
    };
  }, []);

 // 1. ฟังก์ชันจัดระเบียบสิทธิ์และการดึงพิกัดเริ่มต้น
  const setupLocationAndSensors = async () => {
    try {
      setIsLoading(true);

    // ขออนุมัติเข้าถึง Location
      const { status } = await
Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') {
        setHasPermission(false);
        setIsLoading(false);
        return;
      }

      setHasPermission(true);

    // อ่านค่าพิกัดปัจจุบัน
      const location = await Location.getCurrentPositionAsync({
        accuracy: Location.Accuracy.Balanced,
      });

      const coordsObj = {
        latitude: location.coords.latitude,
        longitude: location.coords.longitude
      };

      setCurrentCoords(coordsObj);

    // ดึงที่อยู่ถนนจริงจากพิกัด GPS (Reverse Geocode)
      const reverseResponse = await
Location.reverseGeocodeAsync(coordsObj);
      if (reverseResponse.length > 0) {
        const addr = reverseResponse[0];
        const streetName = addr.street || addr.name || 'ไม่มีระบุถนน';
        const districtName = addr.district || 'ไม่มีระบุอำเภอ';
        const regionName = addr.region || 'ไม่มีระบุจังหวัด';
        setAddress(`📍 ${streetName}, อ.${districtName}, จ.${regionName}`);
      }

    // เริ่มต้นดักฟังข้อมูลเซนเซอร์เขย่า
      startAccelerometerListening(coordsObj);

    } catch (error) {
      console.error(error);
      Alert.alert('ข้อผิดพลาด', 'ไม่สามารถเชื่อมต่อระบบ GPS หรือระบุพิกัดในเครื่องได้');
```

```
    } finally {
      setIsLoading(false);
    }
  };

// 2. ฟังก์ชันจดจำสัญญาณ Accelerometer
  const startAccelerometerListening = (coords: Coords) => {
 // กำหนดรอบการดึงข้อมูลเซนเซอร์ถี่ขึ้น (200 มิลลิวินาที)
    Accelerometer.setUpdateInterval(200);

    accelerometerSubscription.current = Accelerometer.addListener((data)
=> {
      setAccelerometerData(data);

   // ตรวจสอบเงื่อนไขการเขย่าเครื่อง (เช็คความแรงขยับแกน x หรือ y เกิน 1.8)
      if (Math.abs(data.x) > 1.8 || Math.abs(data.y) > 1.8) {
    // บังคับเลื่อนแผนที่วิ่งกลับมาที่จุดพิกัดปัจจุบันของผู้ใช้
        mapRef.current?.animateToRegion({
          latitude: coords.latitude,
          longitude: coords.longitude,
          latitudeDelta: 0.01,
          longitudeDelta: 0.01,
        }, 1000); // สไลด์กล้องแผนที่ใน 1 วินาที
      }
    });
  };

  const stopAccelerometerListening = () => {
    if (accelerometerSubscription.current) {
      accelerometerSubscription.current.remove();
      accelerometerSubscription.current = null;
    }
  };

// 3. ฟังก์ชันปุ่มกดสปอตไลท์ด่วน (Manual Refocus)
  const handleManualCenter = () => {
    if (currentCoords) {
      mapRef.current?.animateToRegion({
        latitude: currentCoords.latitude,
        longitude: currentCoords.longitude,
        latitudeDelta: 0.01,
        longitudeDelta: 0.01,
      }, 800);
    }
  };

// 4. แสดงกรณีสิทธิ์เข้าใช้งานถูกระงับ
  if (hasPermission === false) {
    return (
```

```
        <SafeAreaView style={styles.errorContainer}>
          <View style={styles.errorCard}>
            <Text style={styles.errorEmoji}>🚫</Text>
            <Text style={styles.errorTitle}>ระบบสิทธิ์ตำแหน่งถูกปิดกั้น</Text>
            <Text style={styles.errorDesc}>
        แอปพลิเคชันจำเป็นต้องเข้าใช้งานพิกัด GPS เพื่อทำจัดเรนเดอร์ภาพแผนที่แสดงจุดพิกัดส่วนบุคคล กรุณาเปิดอนุมัติสิทธิ์
ในแอปการตั้งค่า
            </Text>
            <TouchableOpacity
              style={styles.settingsButton}
              onPress={() => Linking.openSettings()}
            >
              <Text style={styles.settingsButtonText}>เปิดหน้าต่างตั้งค่าสิทธิ์แอป</Text>
            </TouchableOpacity>
            <TouchableOpacity
              style={styles.retryButton}
              onPress={setupLocationAndSensors}
            >
              <Text style={styles.retryButtonText}>🔄 ลองขอสิทธิ์ใหม่อีกครั้ง</Text>
            </TouchableOpacity>
          </View>
        </SafeAreaView>
      );
    }

    return (
      <SafeAreaView style={styles.safeArea}>
        <View style={styles.container}>

          {/* แผงควบคุมแสดงรายละเอียดที่อยู่ด้านบนสุด */}
        <View style={styles.addressPanel}>
            <Text style={styles.addressTitle}>พิกัดปัจจุบันของคุณ</Text>
            <Text style={styles.addressText} numberOfLines={2}>
              {isLoading ? '⏳ กำลังค้นหาข้อมูลตำแหน่งพิกัดดาวเทียม...' : address}
          </Text>
            {currentCoords && (
              <Text style={styles.coordsSub}>
                Lat: {currentCoords.latitude.toFixed(5)}, Lng:
{currentCoords.longitude.toFixed(5)}
              </Text>
        )}
          </View>

          {/* ส่วนแสดงแผนที่ระบุพิกัดภูมิศาสตร์ */}
          <View style={styles.mapContainer}>
            {isLoading || !currentCoords ? (
              <View style={styles.loadingWrapper}>
                <ActivityIndicator size="large" color="#38bdf8" />
                <Text style={styles.loadingText}>กำลังจับคู่แผนที่กับสัญญาณดาวเทียม...
```

```
</Text>
            </View>
      ) : (
            <>
              <MapView
                ref={mapRef}
                style={styles.map}
                initialRegion={{
          latitude: currentCoords.latitude,
                  longitude: currentCoords.longitude,
                  latitudeDelta: 0.01,
                  longitudeDelta: 0.01,
                }}
                showsUserLocation={true}
              >
           {/* ปักเข็มหมุดแสดงพิกัดผู้ใช้ */}
                <Marker
                  coordinate={{
                    latitude: currentCoords.latitude,
                    longitude: currentCoords.longitude
                  }}
                  title="ตำแหน่งของคุณ"
                  description="ปักหมุดตรงพิกัดตาม GPS"
        />
              </MapView>

              {/* ปุ่มควบคุมแบบลอยตัว (Floating action buttons over map) */}
              <TouchableOpacity
                style={styles.floatingButton}
                onPress={handleManualCenter}
              >
                <Text style={styles.floatingButtonText}>🎯 ซิงก์กล้องที่จุดคุณ
</Text>
              </TouchableOpacity>

              {/* แถบรายงานค่าเซนเซอร์ความเร่งด้านขวาล่าง */}
              <View style={styles.sensorInfoBox}>
                <Text style={styles.sensorText}>เขย่าเครื่องเบาๆ เพื่อจัดศูนย์กลาง</Text>
                <Text style={styles.sensorTextVal}>
                  X: {accelerometerData.x.toFixed(2)}, Y:
{accelerometerData.y.toFixed(2)}
                </Text>
              </View>
            </>
      )}
        </View>
      </View>
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
  },
  addressPanel: {
    backgroundColor: '#1e293b',
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: '#334155',
  },
  addressTitle: {
    color: '#38bdf8',
    fontWeight: 'bold',
    fontSize: 12,
    textTransform: 'uppercase',
    letterSpacing: 1,
  },
  addressText: {
    color: '#ffffff',
    fontSize: 15,
    fontWeight: 'bold',
    marginTop: 4,
  },
  coordsSub: {
    color: '#94a3b8',
    fontFamily: 'monospace',
    fontSize: 11,
    marginTop: 4,
  },
  mapContainer: {
    flex: 1,
    position: 'relative',
  },
  map: {
    width: '100%',
    height: '100%',
  },
  loadingWrapper: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#0f172a',
  },
  loadingText: {
```

```
    color: '#94a3b8',
    marginTop: 12,
    fontSize: 14,
  },
  floatingButton: {
    position: 'absolute',
    bottom: 20,
    left: 20,
    backgroundColor: '#38bdf8',
    paddingVertical: 12,
    paddingHorizontal: 20,
    borderRadius: 24,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 4 },
    shadowOpacity: 0.3,
    shadowRadius: 5,
    elevation: 6,
  },
  floatingButtonText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  sensorInfoBox: {
    position: 'absolute',
    bottom: 20,
    right: 20,
    backgroundColor: 'rgba(15, 23, 42, 0.85)',
    padding: 10,
    borderRadius: 12,
    borderWidth: 1,
    borderColor: '#334155',
    alignItems: 'flex-end',
  },
  sensorText: {
    color: '#94a3b8',
    fontSize: 10,
  },
  sensorTextVal: {
    color: '#38bdf8',
    fontFamily: 'monospace',
    fontSize: 11,
    fontWeight: 'bold',
    marginTop: 2,
  },
  errorContainer: {
```

```
    flex: 1,
    backgroundColor: '#0f172a',
    justifyContent: 'center',
    padding: 24,
  },
  errorCard: {
    backgroundColor: '#1e293b',
    borderRadius: 24,
    padding: 24,
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
  },
  errorEmoji: {
    fontSize: 48,
    marginBottom: 16,
  },
  errorTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#ffffff',
    marginBottom: 10,
  },
  errorDesc: {
    fontSize: 14,
    color: '#cbd5e1',
    textAlign: 'center',
    lineHeight: 22,
    marginBottom: 24,
  },
  settingsButton: {
    backgroundColor: '#ef4444',
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 12,
    width: '100%',
    alignItems: 'center',
    marginBottom: 12,
  },
  settingsButtonText: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 14,
  },
  retryButton: {
    backgroundColor: 'transparent',
    paddingVertical: 12,
```

```
    width: '100%',
    alignItems: 'center',
    borderWidth: 1,
    borderColor: '#334155',
    borderRadius: 12,
  },
  retryButtonText: {
    color: '#94a3b8',
    fontWeight: 'bold',
    fontSize: 14,
  },
});
```

5. การวิเคราะห์สรุปผลการประเมินปฏิบัติการ

เมื่อติดตั้งและทดลองเปิดแอปพลิเคชัน ให้ผู้เรียนประเมินความสมบูรณ์ดังนี้:

1. หากทำการยกเลิกไม่ยอมเปิด GPS แถบแสดงผลของแอปพลิเคชันต้องดีดสลับมาหน้ากล่องข้อความเตือนภัยสีแดง และกดปุ่ม Settings ได้ทันที

2. เมื่อเปิดบริการ GPS รายการแผนที่ต้องดึงศูนย์กลางกล้องมาโฟกัสพิกัดผู้ใช้และปักหมุดได้อย่างแม่นยำ

3. เมื่อใช้นิ้วเลื่อนแผนที่หนีขอบเขตไปไกล และออกแรงเขย่าตัวเครื่อง (หรือปรับจำลอง Accelerometer ในอุปกรณ์ Emulator) แผนที่ต้องดีดตัวอนิเมตกล้องกลับมาที่จุดพิกัดปัจจุบันโดยอัตโนมัติ

เอกสารอ้างอิง

Expo. (2024). *Location*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/location/

Expo. (2024). *Sensors*. จาก https://docs.expo.dev/versions/v54.0.0/sdk/sensors/

React Native Maps. (2024). *React Native Maps Installation*. จาก https://github.com/react-native-maps/react-native-maps